GitHub 저장소와 Git CLI 실습

01

AI를 활용한 커밋 메시지와 README 작성

2026년 10월 19일 월요일 · 8교시 · 강사 이규영

Python 기초 시간에 만든 작은 프로그램을 Git으로 기록하고 GitHub에 올립니다. 변경 내용을 직접 비교한 뒤 AI로 커밋 메시지와 README 초안을 만들고, 사실과 일치하는지 검증합니다.

> 오늘의 완료 기준: 실행 가능한 Python 프로그램, 의미 있는 커밋 이력, 비밀정보가 제외된 원격 저장소, 실제 코드에 맞는 README, 다시 사용할 AI 지침 파일을 남깁니다.

학습 흐름

01	**저장소 준비**	Git 설정 → 작업 폴더 → 초기 파일 → 첫 커밋
02	**원격 연결**	빈 GitHub 저장소 → push → 별도 폴더에 clone → pull
03	**변경 기록**	로컬 브랜치 → 작은 기능 수정 → diff 확인 → 커밋
04	**AI 문서화**	규칙 파일 → 커밋 메시지 초안 → README 초안 → 직접 검증

단계 | 학생이 수행할 작업

준비물

Windows PC를 기준으로 VS Code, Git for Windows의 Git Bash, Python 3.10 이상, GitHub 계정을 사용합니다. Git 설치와 로그인은 수업 시작 전에 확인하면 좋습니다. AI 도구는 기존 수업에서 사용하는 Codex 또는 GitHub Copilot을 활용합니다.

이 교안의 명령어는 Git Bash 기준입니다. $ 기호 없이 명령어만 입력합니다. 계정명 YOUR_ID는 자신의 GitHub 사용자 이름으로 바꿉니다. Git 결과의 해시·경로·화면 문구는 PC마다 달라질 수 있습니다.

오늘의 범위와 시간표

02

10월 14~16일에 학습한 함수·클래스·예외 처리를 출발점으로 삼습니다. 새 웹 프레임워크를 설치하지 않고 Git과 AI 문서화에 집중합니다.

교시	주제와 50분 배분	완료 증거
1	개념 15분 · 저장소 탐색 20분 · 준비 확인 15분	Git과 GitHub 차이 설명
2	설정 15분 · Python 파일 20분 · ignore 검증 15분	실행 결과와 제외 규칙
3	영역 개념 10분 · 첫 커밋 25분 · diff 15분	초기 이력 2개
4	원격 연결 15분 · push와 clone 20분 · pull 15분	원격 및 복제본 일치
5	브랜치 10분 · 기능 수정 25분 · 상태 점검 15분	기능 브랜치 커밋
6	규칙 10분 · AI 생성 20분 · 검증과 기록 20분	근거와 맞는 메시지
7	지침 파일 15분 · 적용 20분 · 대체 경로 15분	재사용 가능한 지침
8	README 20분 · 종합 미션 20분 · 평가 10분	최종 저장소와 검증표

교시당 50분 수업 후 10분 휴식으로 운영합니다. 마지막 교시 이후 휴식은 운영 일정에 따릅니다. 표의 시간은 수업 운영 제안입니다.

다음 수업과의 연결

10월 20~23일: 같은 저장소에서 FastAPI 설치, 요청·응답 검증, 비동기 처리, API 테스트를 확장합니다. 오늘은 API 엔드포인트·Pydantic·MCP·LangGraph를 구현하지 않습니다.

10월 26일: CRUD와 팀 PR 협업을 진행합니다. 오늘은 개인 브랜치의 생성·전환·원격 업로드까지만 필수로 다룹니다. PR 생성·리뷰·병합·충돌 해결은 다음 수업에서 수행합니다.

> Python 기초 결과물이 이미 있다면 main.py의 프로그램만 기존 결과물로 바꿀 수 있습니다. 이 경우 AI 프롬프트와 README의 실행 결과도 실제 코드에 맞춰 수정하세요.

설치와 인증 확인

03

1 설치가 안 되어 있을 때

Git은 https://git-scm.com/downloads 에서 Git for Windows를 설치하고, VS Code는 https://code.visualstudio.com/download 에서 설치합니다. Windows 설치 도구가 준비되어 있다면 아래 명령도 사용할 수 있습니다. 이 블록만 PowerShell에서 실행합니다.

TERMINAL

```
winget install --id Git.Git -e
winget install --id Microsoft.VisualStudioCode -e
```

설치 후 VS Code와 터미널을 다시 엽니다. VS Code에서 Terminal → New Terminal의 프로필을 Git Bash로 선택합니다. Python은 앞선 수업의 설치 환경을 사용합니다.

TERMINAL

```
git --version
python --version
```

예상 결과: git version 2.x 이상과 Python 3.10 이상이 표시됩니다. python을 찾지 못하면 Windows에서 py --version을 확인하고 이후 python 명령을 py로 바꿉니다.

2 Git 작성자와 GitHub 인증은 별개

git config의 이름·이메일은 커밋 작성자 정보입니다. GitHub 로그인이나 저장소 쓰기 권한을 설정하지 않습니다. VS Code 계정 로그인과 Git CLI 인증도 각각 확인해야 합니다.

HTTPS 인증은 Git Credential Manager의 브라우저 로그인 경로를 사용합니다. 첫 push에서 표시되는 브라우저에 본인 계정으로 로그인합니다. GitHub 계정 비밀번호를 터미널의 Git 비밀번호로 입력하는 방식은 사용하지 않습니다. [3]

3 AI 도구 준비

Copilot 경로는 VS Code에서 사용할 수 있는 Copilot 기능·로그인·조직 정책·사용 한도를 확인합니다. 반짝이 아이콘이 없거나 AI 기능이 차단되어도 수업을 계속할 수 있습니다. Codex 또는 승인된 AI 채팅에 검토한 diff와 규칙을 직접 전달하는 경로를 6~8교시에 제공합니다.

실습용 .env에는 DEMO_ONLY 같은 가짜 값만 사용합니다. 실제 API 키·고객 코드·개인정보를 AI 입력이나 공개 저장소에 넣지 않습니다. 저장소 공개 범위는 교육기관 기준에 따르며 이 교안은 Private로 시작합니다.

Git과 GitHub를 구분하기 04

학습 목표

파일 저장, 스테이징, 커밋, push가 각각 다른 동작임을 설명합니다. GitHub 저장소의 파일 목록·커밋 이력·Code 메뉴를 찾을 수 있습니다.

개념	뜻과 관찰 위치
Git	PC에서 변경 이력을 관리하는 도구. 네트워크 없이 커밋 가능
GitHub	원격 Git 저장소를 호스팅하고 협업을 지원하는 서비스
작업 폴더	에디터에서 수정하는 실제 파일
스테이징 영역	다음 커밋에 포함할 내용을 고른 상태
로컬 저장소	커밋을 기록한 이력. 보통 .git 폴더에 저장
원격 저장소	push로 공유하는 GitHub의 저장소

화면 실습

1. GitHub에 로그인하고 프로필 메뉴의 Your repositories를 엽니다.
2. 기존 실습 저장소가 있으면 파일 목록에서 README를 열고 커밋 이력을 확인합니다. 저장소가 없다면 강사의 예시 화면을 관찰합니다.
3. Code 메뉴에서 HTTPS 주소가 어디에 있는지 확인합니다.
4. 새 저장소 생성 화면 https://github.com/new 을 열어 이름·공개 범위·초기 파일 옵션을 살펴봅니다. 실제 생성은 4교시에 합니다.

생각해 보기

질문: 파일을 저장했는데 왜 GitHub에서는 바뀌지 않았을까요?
답: 저장은 작업 파일만 바꿉니다. 커밋할 파일을 선택하고 커밋한 뒤 push해야 원격에 반영됩니다.

질문: Git은 매번 파일 전체를 중복 복사하나요?
답: Git의 개념 모델은 프로젝트 상태의 스냅샷입니다. 변경 없는 내용은 재사용하며 내부 저장과 전송에는 압축 등의 최적화가 적용됩니다.

> 확인 미션: 저장 → add → commit → push를 본인의 말로 설명하세요. 로컬 저장소를 만드는 git init과 원격을 복제하는 git clone은 시작 경로가 다릅니다. 한 폴더에 둘 다 수행하지 않습니다.

Python 기초 결과물을 준비하기 05

1 작업 폴더와 작성자 설정

아래 명령은 Git Bash에서 실행합니다. mkdir 명령은 아직 같은 이름의 폴더가 없을 때 한 번만 실행합니다. 사용자 이름과 이메일은 예시를 교체하세요. 공개 커밋에 이메일이 노출될 수 있어 GitHub 이메일 설정의 비공개용 주소를 사용할 수 있습니다.

TERMINAL

```
mkdir ai-git-starter
cd ai-git-starter
git config --global user.name "YOUR_NAME"
git config --global user.email "YOUR_EMAIL"
git config --global init.defaultBranch main
git config --get user.name
git config --get user.email
```

VS Code의 File → Open Folder에서 ai-git-starter를 엽니다. 앞으로 명령은 이 폴더 안에서 실행합니다. 전역 설정은 PC의 다른 저장소에도 적용됩니다. 공용 PC에서는 수업 운영 기준을 따릅니다.

2 main.py 생성

VS Code 탐색기에서 main.py를 만들고 다음 코드를 UTF-8로 저장합니다. 외부 패키지 설치 없이 실행할 수 있습니다.

PYTHON

```python
class StudyPlanner:
    def __init__(self, learner: str):
        self.learner = learner

    def plan(self, topic: str) -> str:
        topic = topic.strip()
        if not topic:
            raise ValueError("topic must not be empty")
        return f"{self.learner}: {topic}"


if __name__ == "__main__":
    planner = StudyPlanner("student")
    print(planner.plan("Python basics"))
```

TERMINAL

```
python main.py
```

> 예상 결과: student: Python basics

파일의 역할

StudyPlanner는 클래스, plan은 메서드, strip은 앞뒤 공백 제거입니다. 빈 문자열을 받으면 ValueError를 발생시킵니다. 오늘은 이 코드를 작은 단위로 수정하며 변경 이력을 남깁니다.

추적할 파일과 제외할 파일 06

1 초기 파일 만들기

프로젝트 루트에 .gitignore, .env, README.md를 만듭니다. .gitignore에는 다음 규칙을 입력합니다.

GITIGNORE

```
.env
.env.*
!.env.example
.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
```

.env에는 DEMO_TOKEN=DEMO_ONLY 한 줄만 넣습니다. 실제 인증키는 넣지 않습니다. README.md에는 아래 두 줄을 작성합니다.

MARKDOWN / PROMPT

```
# AI Git Starter
Python study planner for Git practice.
```

2 저장소 초기화와 제외 확인

TERMINAL

```
git init -b main
git status --short
git check-ignore -v .env
git ls-files .env
```

예상 결과: status에는 .gitignore, README.md, main.py가 ?? 상태로 보입니다. check-ignore에는 .env가 제외된 규칙과 위치가 표시됩니다. ls-files .env는 아무것도 출력하지 않아야 합니다.

중요한 예외

.gitignore는 기본적으로 추적하지 않는 파일의 추가를 막습니다. 이미 추적 중인 파일이나 과거 커밋에서 비밀정보를 지워 주지 않습니다. 새 제외 규칙을 추가했어도 git ls-files에 파일이 나오면 추적 중입니다. [1]

실수로 스테이징만 했다면 git restore --staged .env로 내립니다. 단, 첫 커밋 전에는 git rm --cached -- .env를 사용합니다. 이미 추적 중인 파일을 앞으로 제외하려면 git rm --cached -- .env 후 커밋합니다. 로컬 파일은 남지만 과거 이력은 그대로입니다.

실제 키를 한 번이라도 공개하거나 외부로 전달했다면 우선 키를 폐기·재발급하고 강사에게 알립니다. 파일 삭제나 후속 커밋만으로 유출된 키가 안전해지지는 않습니다.

LICENSE는 코드 사용 조건을 설명하는 파일입니다. 오늘은 역할만 확인합니다. 교육기관의 공개 정책과 코드 권리를 확인하기 전 임의의 라이선스를 추가하지 않습니다.

첫 커밋과 상태 기호 읽기

07

1 포함할 파일을 지정하기

TERMINAL

```
git add .gitignore README.md main.py
git status --short
git diff --staged --stat
git diff --staged
```

예상 상태: A .gitignore, A README.md, A main.py. 앞의 A는 스테이징된 새 파일입니다. diff 화면이 길어 (END)가 나타나면 q로 닫습니다. 파일명과 내용에 .env가 없는지 확인합니다.

2 커밋하기

TERMINAL

```
git commit -m "feat: add study planner baseline"
git log --oneline -3
git status
```

예상 결과: 첫 커밋의 짧은 해시와 메시지가 표시됩니다. status에는 nothing to commit, working tree clean과 비슷한 문구가 나옵니다. 이 커밋은 아직 내 PC에만 있습니다.

3 VS Code 화면과 대조

CLI	VS Code에서 관찰
git status	Changes와 Staged Changes를 확인
git add main.py	Changes 파일의 + 버튼
git restore --staged main.py	Staged Changes 파일의 - 버튼
git diff --staged	Staged Changes 파일을 클릭
git commit -m "메시지"	메시지를 입력하고 Commit

파일 저장은 스테이징을 갱신하지 않습니다. 한 번 add한 뒤 다시 수정하면 같은 파일이 Changes와 Staged Changes 양쪽에 나타날 수 있습니다. 커밋에는 마지막으로 add한 내용만 들어갑니다. [2]

확인 미션: git ls-files를 실행해 .gitignore, README.md, main.py가 추적되고 .env는 빠져 있는지 확인하세요. 해시 값은 친구와 달라도 정상입니다.

두 번째 커밋과 안전한 취소

08

1 README의 변경을 눈으로 비교

README.md 끝에 다음 줄을 추가하고 저장합니다.

PROMPT / TEXT

```
Run: python main.py
```

TERMINAL

```
git diff
git add README.md
git diff --staged
git commit -m "docs: add local run command"
git log --oneline -3
```

> 예상 결과: diff에는 Run 줄이 추가된 것으로 보입니다. log에는 docs 커밋과 feat 커밋, 총 2개가 최신순으로 나옵니다.

2 상태 기호 이해

표시	해석
?? file	아직 추적하지 않는 파일
M file	작업 폴더에서 수정했고 스테이징하지 않음
M file	수정 내용이 스테이징됨
MM file	스테이징한 뒤 추가로 수정함
A file	새 파일을 스테이징함

3 실수로 add했을 때

README에 Practice note. 한 줄을 임시로 추가한 뒤 아래 명령을 실행합니다. 첫 커밋 이후의 상황입니다.

TERMINAL

```
git add README.md
git restore --staged README.md
git diff
```

> 예상 결과: 파일 내용은 남고 스테이징만 해제됩니다. 확인 후 에디터에서 방금 추가한 Practice note.만 지우고 저장해 깨끗한 상태로 돌아갑니다.

> git restore --staged는 스테이징 취소입니다. 옵션 없는 git restore 파일명은 작업 내용을 버릴 수 있어 다른 동작입니다. 오늘은 reset --hard, clean -fd, 강제 push를 사용하지 않습니다.

빈 원격 저장소에 처음 올리기

09

1 GitHub에서 저장소 생성

https://github.com/new 에서 이름을 ai-git-starter로 지정합니다. 공개 범위는 Private로 시작합니다. Add README, .gitignore, license 등 초기 파일 추가 옵션은 모두 끕니다. 이미 로컬에서 커밋을 만들었기 때문입니다.

Create repository를 누르고 Code 또는 초기 안내에서 HTTPS URL을 복사합니다. 아래 YOUR_ID를 본인의 GitHub 계정명으로 바꿉니다.

2 연결과 push

TERMINAL

```
git branch --show-current
git remote add origin https://github.com/YOUR_ID/ai-git-starter.git
git remote -v
git push -u origin main
```

예상 결과: main이 업로드되고 origin/main을 추적하도록 설정됩니다. GitHub 화면을 새로고침하면 main.py, README.md, .gitignore가 보입니다. .env는 없어야 합니다.

3 서로 다른 정보 세 가지

항목	확인 방법
현재 브랜치	git branch --show-current → main
원격 URL	git remote -v → 자신의 저장소 주소
추적 관계	git branch -vv → main에 [origin/main] 표시

원격 설정 오류를 만났을 때

remote origin already exists: git remote -v로 기존 주소를 확인합니다. 잘못된 경우에만 아래 명령으로 주소를 바꿉니다. 저장소를 다시 init하지 않습니다.

TERMINAL

```
git remote set-url origin https://github.com/YOUR_ID/ai-git-starter.git
```

Repository not found 또는 403: URL, 로그인 계정, 접근 권한을 확인합니다. VS Code 로그인만으로 Git CLI 권한이 확인된 것은 아닙니다. 인증키를 URL에 넣어 저장하지 않습니다.

확인 미션: GitHub 파일 목록과 로컬 git ls-files를 비교하세요. push는 원격 업로드이며 서비스 배포가 아닙니다. Python 프로그램이 웹에서 실행되는 것은 아닙니다.

clone과 pull로 같은 상태 맞추기

10

1 다른 폴더에 복제하기

원본과 복제본을 구분하기 위해 복제 대상 폴더명을 명시합니다. 원본 폴더 ai-git-starter에서 아래 명령을 실행합니다.

TERMINAL

```
cd ..
git clone https://github.com/YOUR_ID/ai-git-starter.git ai-git-check
cd ai-git-check
git status
git remote -v
python main.py
```

예상 결과: working tree clean, origin URL, student: Python basics가 확인됩니다. clone은 .git 이력과 원격 설정도 가져옵니다. 복제본에서 git init이나 remote add를 다시 할 필요가 없습니다.

2 원격 변경을 내려받기

개인 실습 저장소의 GitHub 화면에서 README.md 편집 버튼을 눌러 Remote sync practice. 한 줄을 추가합니다. 이번 확인 실습에서만 main에 직접 커밋합니다. 복제본 ai-git-check에서 실행합니다.

TERMINAL

```
git switch main
git status --short
git pull --ff-only origin main
git log --oneline -3
```

예상 결과: Fast-forward와 변경 요약이 보이고 README에 추가한 줄이 나타납니다. 이미 받은 경우 Already up to date라고 나옵니다.

3 원본 폴더로 복귀

TERMINAL

```
cd ../ai-git-starter
git status --short
git pull --ff-only origin main
```

status 결과가 비어 있을 때 진행합니다. pull은 현재 체크아웃한 브랜치를 변경합니다. origin main을 지정했다고 자동으로 main으로 전환되지는 않습니다.

--ff-only는 로컬과 원격이 갈라져 자동 병합이 필요한 경우 멈춥니다. divergent branches 또는 non-fast-forward 오류에서는 강제 push하지 말고 상태·이력을 강사와 확인하세요. 팀 PR 병합과 충돌 해결은 10월 26일에 다룹니다.

팀 저장소 접근은 소유자의 초대와 팀원의 수락이 필요할 수 있습니다. 오늘은 화면과 역할만 확인하며 학생 개인 저장소에서 동기화를 완료합니다. 실제 팀 권한 구성은 후속 협업 실습에서 수행합니다.

개인 작업 브랜치 만들기 11

학습 목표

현재 브랜치를 확인하고, 최신 main에서 작업 브랜치를 만들어 수정합니다. 브랜치를 전환했을 때 파일이 어떻게 달라지는지 관찰합니다.

1 시작 상태 확인

TERMINAL

```
git switch main
git status --short
git pull --ff-only origin main
git switch -c feat/study-topic
git branch --show-current
```

> 예상 결과: Switched to a new branch와 feat/study-topic이 표시됩니다. status가 비어 있지 않다면 이전 교시의 미완료 변경을 먼저 확인합니다. 무조건 버리거나 덮어쓰지 않습니다.

2 작은 수정 한 가지

main.py 마지막 줄에서 Python basics를 Git workflow로 바꿉니다. 나머지 클래스와 함수는 유지합니다.

PYTHON

```
if __name__ == "__main__":
    planner = StudyPlanner("student")
    print(planner.plan("Git workflow"))
```

TERMINAL

```
python main.py
git diff -- main.py
git add main.py
git diff --staged -- main.py
```

> 예상 결과: student: Git workflow. diff에서는 문자열 한 줄의 교체만 확인됩니다. 원격 main이나 다른 브랜치는 아직 바뀌지 않았습니다.

3 수정 내용 기록

TERMINAL

```
git commit -m "chore: update demo study topic"
git status --short
```

> 예상 결과: 커밋 성공 후 status가 비어 있습니다. 이것은 새 기능보다 시연 데이터 변경에 가까우므로 이번 예시에서는 chore를 사용합니다. 메시지 규칙의 상세 내용은 6교시에 다룹니다.

> 이미 같은 이름의 브랜치가 있다면 git branch --list로 확인합니다. 새 브랜치가 필요하면 이름 뒤에 -2를 붙이세요. 기존 브랜치로 돌아갈 때는 -c 없이 git switch 브랜치명을 사용합니다.

브랜치 전환과 원격 업로드

12

1 코드가 달라지는지 확인

```
TERMINAL
git switch main
python main.py
git switch feat/study-topic
python main.py
```

> 예상 결과: main에서는 student: Python basics, feat/study-topic에서는 student: Git workflow입니다. 브랜치마다 가리키는 커밋이 다르므로 파일도 달라집니다.

2 내 브랜치를 업로드

```
TERMINAL
git push -u origin feat/study-topic
git branch -vv
git log --oneline --graph --decorate --all -6
```

> 예상 결과: 기능 브랜치의 추적 대상이 origin/feat/study-topic으로 설정됩니다. GitHub의 브랜치 선택 메뉴에서 main과 feat/study-topic을 각각 열어 코드 차이를 확인합니다.

3 다음 교시 시작점

6~8교시는 feat/study-topic에서 계속합니다. main으로 병합하지 않습니다. 변경 내용과 문서를 이 브랜치에 누적하고 마지막에 같은 브랜치로 push합니다.

지금 하는 작업	10월 26일에 연결할 작업
브랜치 생성·전환	팀원별 브랜치 운영
개별 브랜치 push	Pull Request 생성과 코드 리뷰
main과 기능 브랜치의 차이 확인	승인 후 병합과 전체 팀 동기화
깨끗한 작업 폴더 점검	충돌 재현과 안전한 해결

> PR 안내 배너가 보여도 오늘은 생성하지 않아도 됩니다. 브랜치 push만으로 main은 바뀌지 않습니다. 작업이 main에 아직 없다는 것은 이번 수업의 정상 완료 상태입니다.

확인 미션

아래 세 질문에 답하세요.

1. 현재 브랜치 이름은 무엇인가요?
2. main과 달라진 파일은 무엇인가요?
3. push를 했는데 main의 실행 결과가 같은 이유는 무엇인가요?

AI 커밋 메시지의 근거 만들기 13

1 메시지 규칙

기본 형식은 type(scope): 요약입니다. scope는 생략할 수 있습니다. Conventional Commits에서 feat는 새 기능, fix는 오류 수정에 사용합니다. docs·chore·refactor 같은 추가 타입은 팀이 정한 규칙으로 사용합니다. 오늘은 한국어 요약을 권장하며 길이 제한은 팀 규칙으로 정합니다.

타입	이 수업의 예
feat	새 함수나 새 기능 추가
fix	빈 입력 처리 오류 수정
docs	README의 실행 방법 보완
chore	실습용 문구나 설정 정리
refactor	동작을 유지한 구조 개선

2 AI가 검토할 작은 변경 만들기

main.py의 main 블록 바로 위에 아래 함수를 추가합니다. 메인 출력은 그대로 두고, 함수 호출을 별도로 확인합니다.

PYTHON

```python
def normalize_topic(topic: str) -> str:
    return " ".join(topic.split())
```

TERMINAL

```
python -c "from main import normalize_topic; print(normalize_topic(' Git   CLI '))"
git add main.py
git diff --staged --stat
git diff --staged -- main.py
```

예상 결과: Git CLI. diff에는 normalize_topic 함수 추가만 보입니다. 예상 커밋 메시지는 feat(planner): 학습 주제 공백 정리 함수 추가입니다.

AI 커밋 버튼이 어떤 변경을 참고하는지는 도구와 설정에 따라 달라질 수 있습니다. 오늘은 스테이징할 파일을 명시하고, git diff --staged를 직접 확인해 입력 범위를 통제합니다. AI가 변경 이유나 테스트 통과 여부를 자동으로 안다고 가정하지 않습니다.

AI 초안 생성과 수동 검증

14

경로 A VS Code Copilot

소스 제어에서 Staged Changes의 main.py를 엽니다. 커밋 입력창의 Generate Commit Message 아이콘이 보이면 실행합니다. 생성 결과를 바로 커밋하지 말고 아래 확인표로 검사합니다. UI 위치와 표시 여부는 버전·권한·정책에 따라 달라질 수 있습니다.

경로 B Codex 또는 승인된 AI 채팅

로컬 저장소를 읽을 수 있는 도구에는 아래 요청을 사용합니다. 채팅에 붙여넣는 경우에는 git diff --staged 결과를 직접 검토하고 비밀정보가 없는 부분만 함께 제공합니다.

PROMPT / TEXT

```
Read only the staged diff for main.py.
Draft a commit message; do not commit or push.
Use: type(scope): Korean summary
Describe only the implemented change.
Do not claim tests passed without evidence.
Give one title and at most two body bullets.
```

한국어 요청 예시: 스테이징된 main.py 변경만 읽고 커밋 메시지를 작성해 줘. 한국어 Conventional Commits 형식으로 제목 1개와 본문 최대 2개를 제시해 줘. 커밋·push·파일 수정은 하지 마. 구현되지 않은 기능이나 실행하지 않은 테스트를 언급하지 마.

검증 후 직접 기록

☐ 함수가 실제 추가되었는가 ☐ 타입이 변경 성격과 맞는가
☐ API·MCP·비동기 등 없는 기능이 언급되지 않았는가
☐ 테스트를 실행했다는 근거 없는 문장이 없는가

TERMINAL

```
git diff --staged --check
git commit -m "feat(planner): add topic normalization"
git log -1 --format=%B
```

직접 입력하는 CLI 예시는 영문입니다. AI가 제안한 한국어 제목을 확인해 -m 인자에 넣어도 됩니다. 메시지 내용은 실행 결과가 아니라 변경 내용을 설명합니다.

확인 미션: AI 초안에서 본인이 수정한 부분과 수정 이유를 한 줄로 기록하세요. AI를 사용할 수 없다면 같은 규칙으로 직접 작성하고 수업을 완료합니다.

재사용할 커밋 지침 저장하기 15

1 도구와 분리된 규칙 파일

프로젝트에 .github 폴더를 만들고 commit-instructions.md를 생성합니다. 이 파일은 커밋 생성 규칙의 원본입니다. 파일을 저장한 것만으로 모든 AI 도구가 자동 적용하는 것은 아닙니다.

MARKDOWN / PROMPT

```
# Commit message rules

- Use type(scope): summary.
- Write the summary in Korean.
- Use feat, fix, docs, refactor, or chore.
- Describe only the staged diff.
- Use at most two body bullets.
- Do not invent implementation reasons.
- Do not claim unverified test results.
- Never include secrets.
- Suggest text only; do not commit or push.
```

2 Copilot 설정이 지원되는 환경

.vscode 폴더에 settings.json을 만듭니다. 이미 파일이 있다면 최상위 JSON 객체 안에 항목만 합칩니다. 기존 설정을 통째로 덮어쓰지 않습니다.

JSON

```
{
  "github.copilot.chat.commitMessageGeneration.instructions": [
    {
      "file": ".github/commit-instructions.md"
    }
  ]
}
```

이 설정은 VS Code Copilot의 커밋 메시지 생성용입니다. Codex 설정과는 다릅니다. 공식 문서는 file 또는 text 속성을 갖는 배열을 허용합니다. 실험적 설정 여부와 실제 동작은 사용 버전에서 확인합니다. [5]

3 지침을 함께 기록

TERMINAL

```
git add .github/commit-instructions.md .vscode/settings.json
git diff --staged --check
git diff --staged --stat
```

> 지침 공유는 출력 형식의 일관성을 높이는 방법입니다. 로그인·확장·정책·버전이 다르거나 AI가 규칙을 따르지 않으면 결과가 달라집니다. 팀원 모두에게 100% 동일한 출력이나 자동 적용을 보장하지 않습니다.

자동 적용을 검증하고 대체하기 16

1 적용 시험

Copilot의 커밋 메시지 생성 기능을 실행합니다. 한국어 요약, 올바른 타입, 변경된 파일 범위를 확인합니다. 이번 변경은 규칙·설정 파일 추가이므로 chore(config)와 같은 범위가 자연스럽습니다.

기대 초안 예: chore(config): 커밋 메시지 작성 지침 추가. 결과는 모델과 환경에 따라 달라집니다. 영어로 나오거나 파일을 놓쳤다면 지침 파일 경로, JSON 구문, Copilot 로그인 및 사용 가능 상태를 확인합니다.

2 Codex의 명시적 지침 경로

Copilot 설정이 적용되지 않는 환경에서는 다음 요청을 사용합니다. 파일을 읽는 기능이 없는 채팅에서는 규칙 파일 내용과 검토한 staged diff를 함께 붙입니다.

PROMPT / TEXT

```
Read .github/commit-instructions.md.
Apply it to the current staged diff only.
Return a commit message draft and the file list
used as evidence. Do not change files.
Do not run git commit or git push.
```

한국어 요청 예시: .github/commit-instructions.md를 읽고 현재 스테이징 변경에 적용해 줘. 커밋 메시지 초안과 근거로 읽은 파일 목록을 제시해 줘. 파일 변경, 커밋, push는 하지 마.

3 결과 확인 후 커밋

TERMINAL

```
git diff --staged
git commit -m "chore(config): add commit message instructions"
git status --short
```

재사용 가능한 자동화의 의미

이번 자동화는 입력 변경분 + 저장한 작성 규칙 → AI 초안 → 사람의 검증 → 커밋으로 이어지는 반자동 작업입니다. Git 커밋을 자동 실행하는 hook이나 CI를 설치하는 수업은 아닙니다.

> 확인 미션: 지침 파일의 본문 요약 수를 최대 2개에서 1개로 바꾸고, 변경 diff에 대한 초안을 다시 요청해 보세요. 결과를 비교한 뒤 최종 규칙을 결정하고 변경했다면 별도로 커밋합니다. 이 항목은 여유가 있을 때 수행합니다.

실제 코드로 README 만들기 17

1 README에 필요한 내용

제목과 목적, 현재 기능, Python 버전, 실행 명령, 예시 결과, 파일 역할, 보안 주의사항을 담습니다. 아직 구현하지 않은 API·MCP·자동 복구·배포 기능은 현재 기능으로 적지 않습니다.

2 재사용 프롬프트

아래 본문을 AI에 전달합니다. 로컬 파일 읽기가 불가능한 채팅에서는 main.py, .gitignore와 현재 README의 내용을 직접 제공합니다. .env 내용은 제공하지 않습니다.

PROMPT / TEXT

```
Read main.py, README.md, and .gitignore only.
Update README.md from implemented code.
Include purpose, Python version, file roles,
run command, expected output, and secret handling.
State that third-party packages are not required.
Document normalize_topic as a helper function.
Do not invent API endpoints or MCP integration.
Do not read .env or include any secret values.
Edit README.md only. Do not commit or push.
Mark unrun commands as examples, not verified results.
```

한국어 실행 지시: 위 조건을 지켜 한국어 README를 작성해 줘. 현재 브랜치에서 main.py를 실행했을 때의 출력과 일치시켜 줘. 구현되지 않은 기능은 현재 기능으로 적지 마.

3 Copilot 프롬프트 파일은 선택 실습

지원되는 VS Code Local 세션에서는 .github/prompts/create-readme.prompt.md에 아래 헤더와 위 본문을 함께 저장한 뒤 /create-readme 또는 Run Prompt로 실행할 수 있습니다. agent 필드를 사용합니다. [6]

MARKDOWN / PROMPT

```
---
agent: agent
description: Write README from the implemented Python code
---
```

현재 공식 문서상 .prompt.md는 Local harness에서 지원됩니다. 다른 harness에서 파일이 보이지 않으면 위 본문을 채팅에 직접 입력하는 공통 경로로 진행합니다. 세션마다 지원 범위를 먼저 확인하며, 파일 생성만으로 자동 실행되지는 않습니다.

README 검증과 종합 미션

18

1 예상 README의 핵심 부분

MARKDOWN / PROMPT

```
# AI Git Starter

A Python study planner for Git practice.

## Requirements
Python 3.10 or later; no third-party packages.

## Run
```bash
python main.py
```

Expected output on feat/study-topic:
```text
student: Git workflow
```

Notes
normalize_topic collapses repeated whitespace.
Do not commit .env or real credentials.
```

실제 README는 한국어로 작성해도 됩니다. 정규화 함수는 별도 helper이며 메인 흐름에서 호출하지 않는다는 점을 설명하면 더 정확합니다. 없는 requirements.txt 설치 명령을 넣지 않습니다.

2 검증하고 기록하기

TERMINAL

```
python main.py
python -c "from main import normalize_topic; assert normalize_topic(' A   B ') == 'A B'"
git diff -- README.md
git add README.md
git diff --staged --check
git commit -m "docs: align README with the current program"
git push origin feat/study-topic
git status --short
```

프롬프트 파일도 만들었다면 README 커밋 전에 파일명을 명시해 함께 add합니다. assert 명령은 성공 시 아무것도 출력하지 않고, 실패 시 AssertionError가 납니다.

> 종합 미션: GitHub에서 feat/study-topic을 선택해 README를 확인하고, 별도 폴더로 해당 브랜치를 clone하여 README의 실행 명령을 그대로 따라 합니다. main은 아직 Python basics 상태이며 PR 병합은 다음 수업에서 진행합니다.

완료 점검과 제출 기준

19

새 폴더에서 재현하기

현재 ai-git-starter 폴더에서 다음 명령을 실행합니다. ai-git-final-check 폴더가 이미 있으면 다른 새 이름을 사용합니다.

TERMINAL

```
cd ..
REPO_URL=https://github.com/YOUR_ID/ai-git-starter.git
git clone --branch feat/study-topic "$REPO_URL" ai-git-final-check
cd ai-git-final-check
python main.py
python -c "from main import normalize_topic; assert normalize_topic(' A   B ') == 'A B'"
git ls-files .env
git status --short
```

기대 결과: student: Git workflow. assert는 출력 없이 종료. ls-files .env와 status --short는 출력 없음. 폴더를 복제했으므로 .env 자체도 없어야 정상입니다.

제출 항목

1. 저장소 URL과 작업 브랜치 feat/study-topic
2. git log --oneline -8 결과
3. 새 복제본의 실행 결과
4. AI 초안에서 고친 부분과 이유 1~2문장
5. README와 지침 파일의 저장소 내 경로

Private 저장소는 수업에서 지정한 제출 방법으로 접근 권한을 처리합니다. 비밀번호·토큰은 제출하지 않습니다.

평가 항목	배점	확인 기준
저장소·원격	20	URL·origin·브랜치를 올바르게 구분
변경 기록	25	의미 있는 커밋과 깨끗한 작업 상태
제외 규칙	20	.env 미추적 및 제외 규칙 근거
AI 메시지	15	diff와 일치, 근거 없는 주장 없음
README 재현	20	복제본에서 명령과 결과 일치

총 100점. AI 접근이 불가능해 직접 작성한 경우에도 같은 품질 기준으로 평가합니다. 커밋 횟수나 해시 값 자체를 정답으로 평가하지 않습니다.

자주 발생하는 오류와 대응

20

증상	확인과 해결
git을 찾을 수 없음	Git 설치 후 VS Code와 터미널을 다시 실행. Git Bash 프로필 확인
not a git repository	pwd로 현재 경로 확인. 원본 또는 clone한 저장소 폴더로 이동
Author identity unknown	user.name과 user.email 설정 확인. 계정 로그인 오류와 구별
nothing to commit	파일 저장 여부, git diff, git diff --staged 확인
src refspec main does not match	첫 커밋 유무와 현재 브랜치 확인. git log와 git branch 실행
remote origin already exists	git remote -v 확인 후 잘못된 주소만 set-url로 수정
Repository not found 또는 403	주소·인증 계정·초대 수락·쓰기 권한 확인
non-fast-forward 또는 divergent	git status와 git log 확인. --ff-only가 멈췄다면 강사와 이력 점검. 강제 push 금지
unrelated histories	로컬 init과 README가 있는 원격을 따로 만든 상황인지 확인. --allow-unrelated-histories를 무작정 사용하지 않음
would be overwritten by checkout	미완료 변경 확인 후 커밋하거나 복사해 보존. 무조건 restore하지 않음
clone 대상 폴더가 이미 존재	비어 있는 새 폴더명을 지정. 기존 폴더 삭제로 해결하지 않음
AI 아이콘 또는 명령 없음	로그인·권한·버전·세션 확인. 검토한 diff와 프롬프트 직접 입력
AI가 없는 기능을 설명	실제 코드와 diff를 근거로 다시 요청하고 직접 수정

해결 원칙: 오류 메시지를 읽고 git status, 현재 경로, 현재 브랜치, 원격 URL을 먼저 확인합니다. 오류를 지우기 위한 저장소 삭제·강제 push·파괴적 초기화는 수행하지 않습니다.

명령어 빠른 참조

21

목적	명령
현재 폴더와 브랜치	pwd / git branch --show-current
상태와 이력	git status --short / git log --oneline -8
작업 변경 확인	git diff -- 파일명
스테이징 변경 확인	git diff --staged -- 파일명
파일 선택	git add 파일명
스테이징 취소	git restore --staged 파일명
커밋	git commit -m "type: summary"
새 브랜치 생성·이동	git switch -c 브랜치명
기존 브랜치 이동	git switch 브랜치명
원격 주소 확인	git remote -v
처음 업로드·추적 설정	git push -u origin 브랜치명
이후 업로드	git push origin 브랜치명
복제	git clone URL 새폴더명
현재 브랜치에서 main 동기화	main으로 전환 후 git pull --ff-only origin main
제외 규칙 확인	git check-ignore -v .env
추적 여부 확인	git ls-files .env

표에서 / 기호는 서로 다른 명령을 나열한 구분자입니다. 한 줄에 함께 입력하지 않습니다. 파일명·URL·브랜치명은 실제 값으로 바꿉니다.

다음 수업에 가져갈 상태

main: Python 기초 프로그램과 Git 시작 이력.
feat/study-topic: 주제 변경, 정규화 함수, AI 지침, 최신 README.
10월 20일에는 강사가 지정한 기준 브랜치에서 FastAPI를 시작합니다.
10월 26일에는 준비된 변경을 예제로 PR·리뷰·병합을 연습할 수 있습니다.

공식 문서와 확인 범위 22

참고 문서는 2026년 10월 18일 확인했습니다. UI와 AI 기능은 업데이트될 수 있으므로 수업 직전 학교 PC의 버전·계정·정책에서 한 번 더 확인합니다. 명령 실행 결과의 해시와 절대 경로는 환경마다 다릅니다.

원본 커리큘럼

https://docs.google.com/spreadsheets/d/1rAEMTPrsFbHV2Gwy0HjsUupiDJiWYuMlqddt8m-4c5Q/edit

[1] Git gitignore

https://git-scm.com/docs/gitignore

[2] VS Code Staging and committing changes

https://code.visualstudio.com/docs/sourcecontrol/staging-commits

[3] GitHub Caching credentials

https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git

[4] GitHub flow

https://docs.github.com/en/get-started/using-github/github-flow

[5] VS Code Custom instructions

https://code.visualstudio.com/docs/agent-customization/custom-instructions

[6] VS Code Prompt files

https://code.visualstudio.com/docs/agent-customization/prompt-files

[7] OpenAI Codex Prompting Guide

https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide

[8] Conventional Commits specification

https://www.conventionalcommits.org/en/v1.0.0/

용어와 실습의 구분

커밋 지침과 README 프롬프트는 이 수업을 위해 작성한 실습 예시입니다. Copilot의 자동 기능과 Codex에 명시적으로 요청하는 방법을 구분했습니다. 로컬 CLI의 생성·커밋·브랜치·원격 복제 흐름과 Python 예제는 검증하며, 개인 계정의 GitHub 인증과 AI 출력은 학생 환경에서 확인합니다. [7]은 프롬프트 참고 문서이며 특정 모델의 사용을 요구하지 않습니다.